

# Week 7

## Introduction to Routing

How a network of 75 000 independent networks finds a path for every packet — with nobody in charge.

University of Canterbury · Yinuo Zhang · [yinuo.zhang@canterbury.ac.nz](mailto:yinuo.zhang@canterbury.ac.nz)

QR + short link – added when hosted

0:00

Welcome to Term 4. This web page *is* the slide deck: same link on your laptop, works on anything. Three lectures this week take us from “what is routing?” to the algorithms running inside every router. House style: I'll ask quick questions as we go — answer on your device or just think; they're anonymous and ungraded.

## What you already know

### Physical · L1

Signals, modulation, digital encoding, synchronisation.

### Link · L2

MAC, Ethernet framing, switches & bridges.

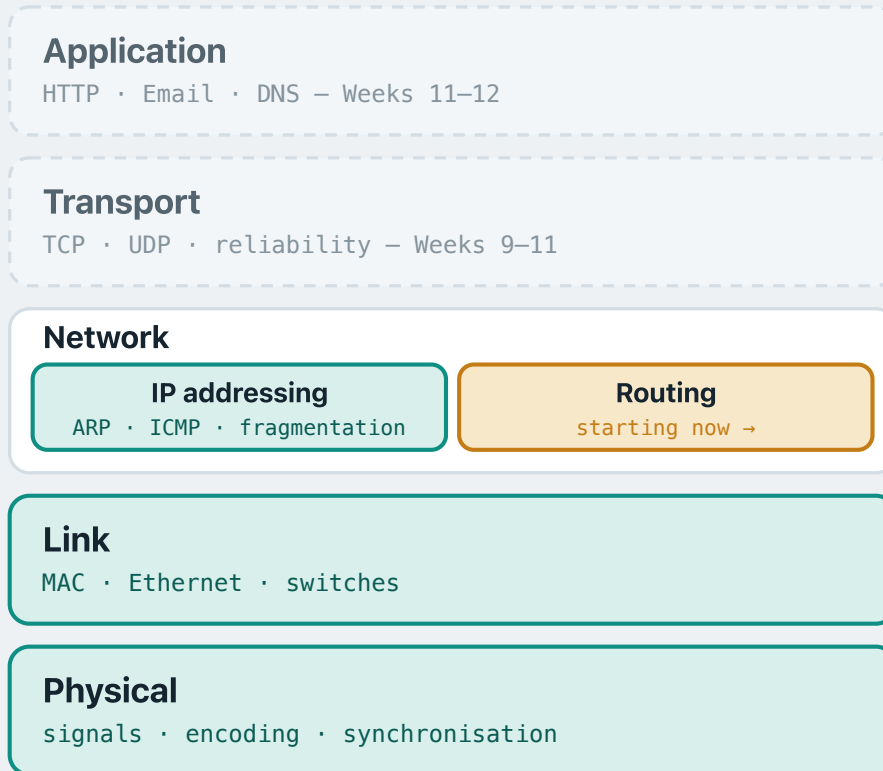
### Network · L3, so far

IP addressing, ARP, ICMP, fragmentation.

**1 min** Sixty seconds, no more — this is a recap, not a re-teach. Name each card, don't re-derive it; if a term draws a blank, point students to last term's notes rather than stopping to re-explain.

ZOOM OUT

## Where this fits — the whole stack



- Done — Term 3
- Starting now — Week 7
- Still ahead — Weeks 8–12

This is the same TCP/IP model from Term 3 — five layers, Session and Presentation folded into Application, exactly as you saw it before.

Everything below the line works. **Today, the Network layer starts doing something cleverer than addressing.**

**3 min** Two minutes, no more. Point at the diagram, don't read every label aloud. The teal bands are settled; the amber cell is the whole reason we're here today. Say explicitly that this is the same five-layer model from Architectures & Basics — it's a recap, not a new framework.

Lecture 1 of 3

# The Routing Problem

---

What routing actually is, how the Internet is stitched together from thousands of independent networks, and why finding a path becomes a problem we can solve with mathematics.

1 · The Routing Problem

2 · Link-State Routing

3 · Distance-Vector & the Real Internet

4 min

Today's title card. One continuous story across the three lectures; this page remembers where we stop and resumes there next time.

# Who tells the packets where to go?

---

A packet leaving this room reaches Sydney in about 21 milliseconds, crossing at least five different organisations on the way. Nobody wrote that path down in advance. **So who — or what — decided?**

**5 min** Let the first sentence land, then the reveal: nobody sat down and wrote out that Sydney path. Today's question is how the real Internet does that job by itself. Let's look at the evidence first.

## Tracing a path from this room

```
$ traceroute www.canterbury.ac.nz
traceroute to adobe-aem.map.fastly.net (146.75.15.10), 18 hops max
 1  10.193.254.254          3.1 ms
 2  10.181.3.201           5.3 ms
 3  * * *
 4  210.7.37.89            10.5 ms
 5  as38022.chc.ix.nz       4.9 ms ← Christchurch Internet Exchange
 6  * * *
 7  * * *
 8  * * *
    ...the destination never answers – yet the site loads fine.
```

[▶ Run](#)[Skip to end](#)

recorded 17 Jul 2026 on the UC network — run it live if the room's network allows

Two campus routers `10.x.x.x` → the **Christchurch Internet Exchange** → then silence: routers that *choose* not to answer. And the destination isn't UC at all — it's a company called Fastly. (Both stories return later in the course.)

**9 min** Run live if the venue network permits; otherwise press Run — it's a replay of this exact command. Point at: (1) private 10.x campus hops; (2) `as38022.chc.ix.nz` — our packets reach the Christchurch exchange within ~5 ms; (3) the stars: traceroute relies on routers volunteering replies, many are firewalled and don't — a one-line aside; (4) the resolved name — `canterbury.ac.nz` is served by a CDN. Don't explain CDNs yet; park it.

## Distance is measurable — usually

```
$ ping -c 3 www.canterbury.ac.nz
round-trip min/avg/max = 5.8 / 7.5 / 10.4 ms
$ ping -c 3 www.sydney.edu.au
round-trip min/avg/max = 20.5 / 20.8 / 21.5 ms
$ ping -c 3 www.ox.ac.uk
round-trip min/avg/max = 9.5 / 10.4 / 11.2 ms    (?!)
```

[▶ Run](#)[Skip to end](#)

Sydney: ~2100 km in 21 ms round trip — **physically plausible**. Someone routed every hop across the Tasman.

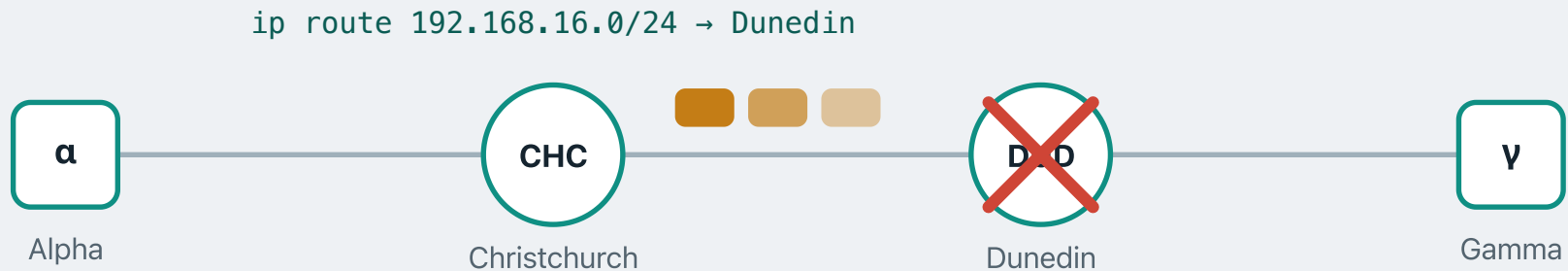
Oxford: ~19000 km in 10 ms — **physically impossible** (light in fibre needs ~190 ms).

Something *near us* answered for Oxford. Hold that thought until the application-layer lectures later this term.

**12 min** The Sydney number is the honest hook: real distance, real routing. The Oxford number is a planted cliffhanger — physics says it cannot be Oxford answering (~9500 km each way  $\Rightarrow \geq 95$  ms one-way in fibre). Promise the resolution in the Web lectures (CDNs). Re-run live before lecture; numbers drift.

## THE TROUBLE WITH FIXED ROUTES

# A route fixed by hand cannot react



Christchurch keeps forwarding into the void — no reaction, no reroute

A static route describes **one moment** of the network. The network doesn't stay in that moment.  
**The rest of this week: how a network routes itself — and heals.**

**15 min** Motivate the gap: a route written by hand keeps forwarding even after the far end dies — the network can't notice or heal itself. Name the week's promise. This bridges into the definition section.



# What routing is

---

A definition, and three jobs any real solution must do.

**16 min** Fast transition slide.

## DEFINITION

# Routing decides the paths packets take

**Routing** is the **network-wide** process that determines the end-to-end paths packets take from source to destination.

*“A **name** indicates what we seek.*

*An **address** indicates where it is.*

*A **route** indicates how we get there.”*

— Jon Postel, RFC 791 (1981), the document that defined IP

**18 min** Emphasise "network-wide": no single router can decide a path alone — its choices only make sense if everyone else's agree. The Postel triple maps onto things they know: DNS names (Term 3), IP addresses (Term 3), routes (now).

## REQUIREMENTS

# Three jobs a self-routing network must do

### 1 · discover

Learn who is out there  
and what each link  
costs — without being  
told.

### 2 · compute

Turn that knowledge  
into best paths — a  
forwarding table per  
router.

### 3 · react

Links fail, costs  
change, routers die.  
Recompute —  
automatically, quickly.

Fixed, hand-written routes do none of these — which is exactly the specification for what comes next.

**20 min** These three verbs structure the next two lectures: link-state and distance-vector are just two philosophies of \*discover + compute\*, and their real difference shows up in \*react\*. Plant that.

# Routing and forwarding: two different jobs

---

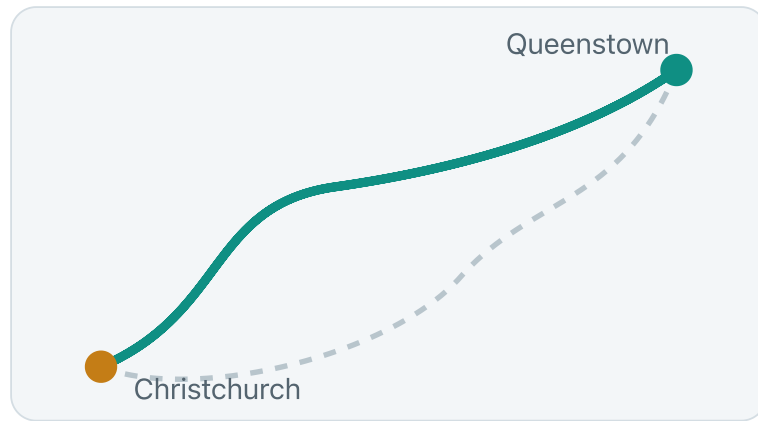
One happens before, network-wide, slowly. The other happens per packet, locally, in nanoseconds.

21 min

The single most-confused pair of words in this course. Spend the analogy properly.

## ANALOGY

# Planning the trip vs. reading the signs



**Routing** — plan the whole path, *before* the trip.

Compares alternatives · has the full map · unhurried



**Forwarding** — one decision per junction, *during* the trip.

No alternatives weighed · looks at one sign · at full speed

**24 min** The sign doesn't know your journey — it only maps destinations to exits. That's precisely a forwarding table. And someone had to decide, earlier and centrally-ish, what to paint on every sign so journeys work end-to-end — that's routing.

FORMALLY

## Control plane, data plane

### CONTROL PLANE · routing



forwarding table  
prefix → next hop  
timescale: seconds–minutes



### DATA PLANE · forwarding



every packet · nanoseconds  
often dedicated hardware

**27 min** Two planes, two timescales, two silicon budgets. Routing tolerates being slow (up to ~minutes); forwarding cannot. This division of labour is why each can be engineered well.

## The forwarding table — you’ve already built one

| DESTINATION PREFIX | NEXT HOP     | INTERFACE |
|--------------------|--------------|-----------|
| 192.168.13.0/24    | 192.168.12.2 | enp0s8    |
| 192.168.16.0/24    | 192.168.13.2 | enp0s8    |
| <b>default</b>     | 192.168.11.1 | enp0s3    |

Minimum contents: a **next hop** and an **interface** for each reachable prefix. Consulted for every packet; rewritten rarely.

```
$ ip route
default via 192.168.11.1 dev enp0s3
192.168.11.0/24 dev enp0s3 proto kernel
scope link src 192.168.11.128
```

A host’s forwarding table — just two entries.

**Routing, in every form we meet from now on, is just: who writes this table, and how.**

**30 min** Demystify: the object at the centre of the whole week is a small, plain table. Every algorithm from here on is a strategy for populating it automatically.

## Routing or forwarding?

A packet arrives; the router looks up its destination prefix and sends it out interface 2.

Routing

Forwarding

Routers exchange messages about link costs and recompute their tables.

Routing

Forwarding

An administrator adds a route by hand: `ip route add default via 192.168.11.1`

Routing

Forwarding

A table entry is applied to a packet in 20 ns by dedicated hardware.

Routing

Forwarding

Anything that *writes* the table — even by hand — is routing. Anything that *reads* it per packet is forwarding.

Reveal

QR / poll link placeholder

**33 min** ~90 seconds. Item 3 is the trap worth discussing: manually writing a route *is* routing — static routing. The plane is defined by the job, not by who does it.



# The Internet is a network of networks

---

Before any algorithm: who owns the routers, and why that splits routing in two.

**34 min** Structure before algorithms — the ownership structure decides which algorithms are even allowed to run where.

# The unit of the Internet: the AS

An **autonomous system** is a set of routers and networks...

managed by a **single organisation**

running a **common routing protocol** internally

**connected** — except in times of failure

Each AS has a number — an **ASN**. There are ~75 000 active ones. The Internet is what happens *between* them.

**36 min** "Autonomous" is the operative word: inside its borders an organisation answers to nobody — its network, its rules, its choice of protocol. Next slide makes it concrete with the AS they're sitting in.

CONCRETELY

## The AS you are inside right now

|               |                                  |
|---------------|----------------------------------|
| AS handle     | <b>AS9432</b>                    |
| ASN name      | CANTERBURY-AS                    |
| organisation  | University of Canterbury         |
| registry      | APNIC (Asia-Pacific)             |
| IPv4 prefixes | 132.181.0.0/16 · 202.36.178.0/23 |

asnlookup.com, retrieved July 2026

Your laptop's campus address lives inside 132.181.0.0/16 — a prefix the University owns and announces to the rest of the

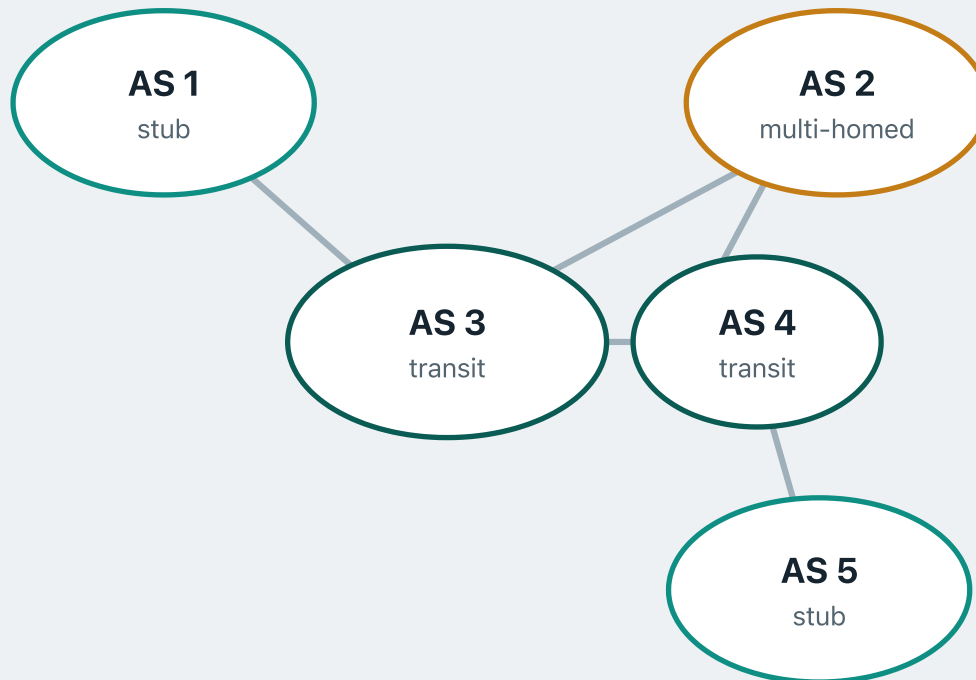
Internet.

Inside AS9432: UC's network team decides everything.

**Outside: nobody owes UC anything — every path is a negotiation.**

**38 min** If the venue network allows, do the lookup live (asnlookup.com or bgp.he.net → AS9432); the table is the recorded fallback. Students enjoy discovering the university is a first-class citizen of the Internet with its own /16.

## Three kinds of AS — by how traffic may flow



**Stub** — one connection to one provider.

AS 1, AS 5 · e.g. this university

**Multi-homed** — several providers, but carries **no through-traffic**.

AS 2 · e.g. a bank that can't afford downtime

**Transit** — connects ASes and **carries others' traffic**. That service is the product.

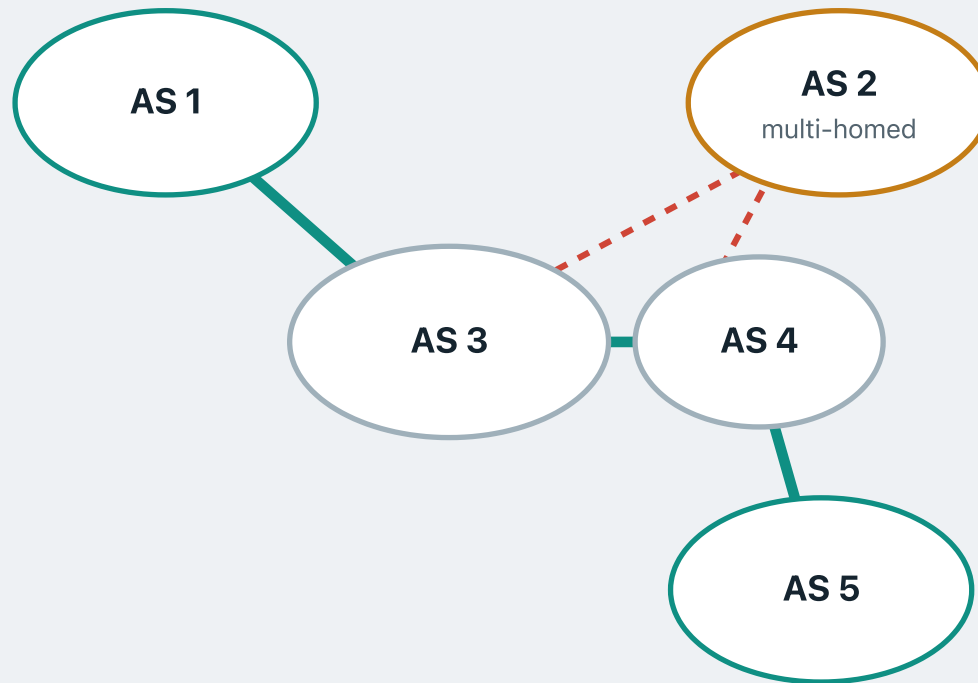
AS 3, AS 4 · e.g. One NZ, Verizon

Netflix moves enormous traffic — but sells video, not carriage: *not* a transit AS.

**42 min** As each card reveals, the matching blobs tint (stub→teal, multi-homed→amber, transit→green-ish teal). The Netflix contrast prevents the common "big company = transit" error. Money defines the categories: stubs pay, transit gets paid, multi-homed pays twice for insurance.

## PREDICT

May AS 1's traffic to AS 5 pass *through* AS 2?



Yes

No

**No.** AS 2 bought connectivity for *its* own traffic — carrying AS 1 → AS 5 through its network would mean paying twice to move a stranger's packets.

The traffic flows **AS1 → AS3 → AS4 → AS5**, through the ASes *paid* to carry it.

Inter-AS routing isn't about shortest — it's about **policy**: who pays whom.

**45 min** Hands up or tap. On reveal the AS2 links cross out and the transit path lights teal. This is the single most important inter-AS idea: topology permits, policy decides. BGP (later this week) exists to express exactly this.

# One Internet, two routing problems

## intra-AS · “interior”

Routing **within** an AS.

- one owner → optimise freely for performance
- protocols: RIP, OSPF, IS-IS

## inter-AS · “exterior”

Routing **between** ASes.

- many owners → policy and money rule
- protocol: BGP — one, for the whole world

These lectures build the algorithms that power the **interior**; the exterior returns later this week.

**47 min** Name-drop only — RIP/OSPF/BGP get their proper treatment in the final lecture this week. The students just need the map.

## What kind of AS?

A university runs its own AS and buys *all* its connectivity from a single ISP, reselling access to its students at a flat rate.

A — Transit: student traffic passes through it

B — It's not an AS: it isn't an ISP

C — Multi-homed: it connects thousands of students

D — Stub: one connection to one provider

Counting *connections to other ASes* is all that matters: one provider  $\Rightarrow$  stub. Students are hosts inside the AS, not ASes.

**Reveal**

**49 min** Distractor C is the common miscue (confusing many *\*users\** with many *\*AS links\**). If time is tight, do this one verbally.

# Turning networks into mathematics

---

Strip away cables and organisations, and routing becomes a problem we can solve exactly.

**50 min** Lecture 1 usually ends around here; the section is self-contained if it spills to Lecture 2. If running late, this section header is a clean stopping point — the week's material resumes exactly here.



## THE MODEL

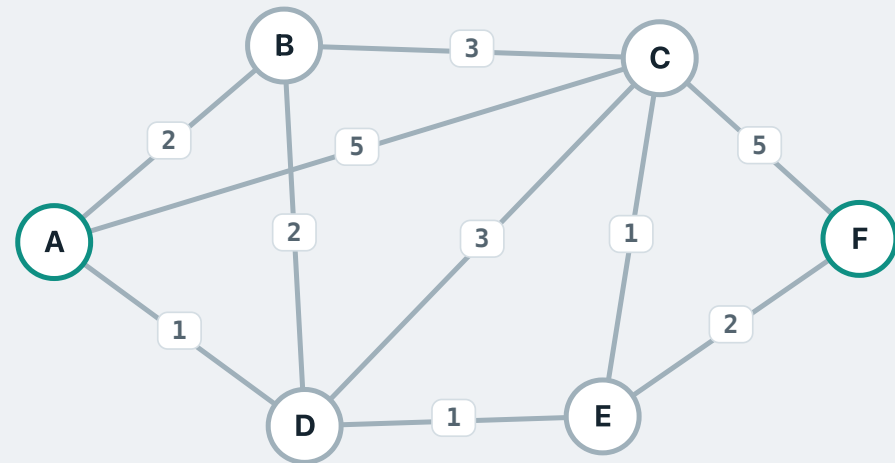
# A network is a weighted graph

routers → **vertices**

links → **edges**

link metrics → **edge weights**  $c(x, y)$

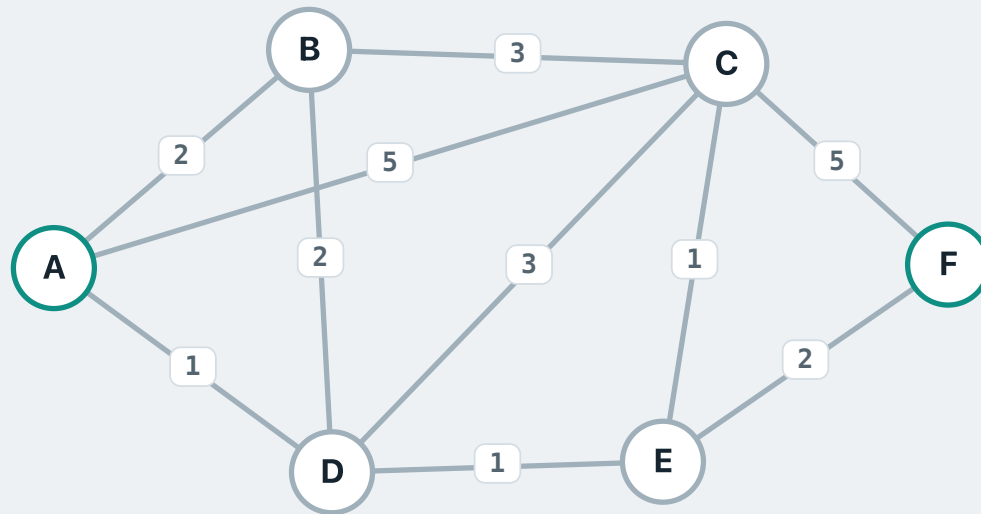
the routing problem → **find least-cost paths**



**52 min** The graph on the right is the week's companion graph — the same six routers reappear in the Dijkstra lecture, in the interactive visualizer, and in the distance-vector section. Everything from here is mathematics on this picture.

## EXPLORE

# Find the cheapest path $A \rightarrow F$



$A \cdot C \cdot F$

$A \cdot B \cdot C \cdot F$

$A \cdot D \cdot E \cdot F$

Cheapest?

☐ set every weight to 1 (hop count)

**55 min** Invite guesses before touching buttons — the eye is drawn to the "direct-looking"  $A \cdot C \cdot F$ , which costs 10. Cheapest is  $A \cdot D \cdot E \cdot F = 4$ : intuition fails even at six nodes, which is why we need an algorithm (tomorrow). Then tick the checkbox: under hop count the winner flips to  $A \cdot C \cdot F$ . **The metric decides what "best" means** — delay, 1/bandwidth, money, or hops. The algorithm doesn't care.

## Four ways to fill a forwarding table

### static

A human writes the routes.

✗ blind to change — cannot react

### random

Forward each packet to a random neighbour (not the one it came from).

✗ arrives eventually — wildly wasteful

### flooding

Send every packet to every neighbour.

△ guaranteed & explosive — **remember it:** it returns tomorrow as link-state's messenger

### adaptive

Measure, compute, react as conditions change.

✓ the only survivor — where this week is headed

**58 min** If this is the Lecture 1 endpoint, stop after this slide and open with a recap next lecture. Play it as an elimination game. Flooding's "remember me" matters: link-state advertisements are delivered by controlled flooding, so the silly strategy becomes a building block.

# Algorithms, protocols, and the two great families

---

Every routing system on Earth descends from one of two ideas.

- Final stretch of the lecture-1 material.

A DISTINCTION THAT MATTERS

# Routing algorithm $\neq$ routing protocol

## algorithm

Solves the least-cost-path problem under **ideal assumptions**: the graph is known, nothing changes, nothing fails.

Dijkstra 1959 · Bellman–Ford 1956-58 — both older than the Internet

## protocol

Embeds the algorithm in **reality**: distributed across routers, exchanging messages that are *delayed*, *lost*, *reordered* — while nodes crash.

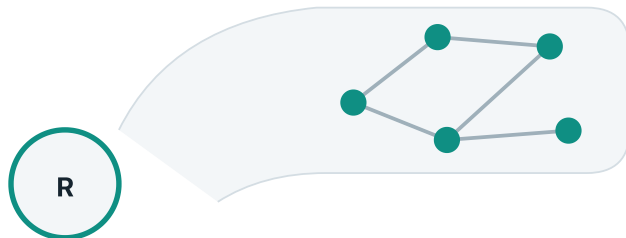
RIP, OSPF, BGP — engineering around the algorithm

Most of what's hard (and interesting) in routing lives in the gap between the two.

— Both algorithms predate ARPANET — the maths was waiting for the network. The protocol column is where Wednesday's failure stories (routing loops, count-to-infinity) come from.

## Get the whole map — or ask your neighbours

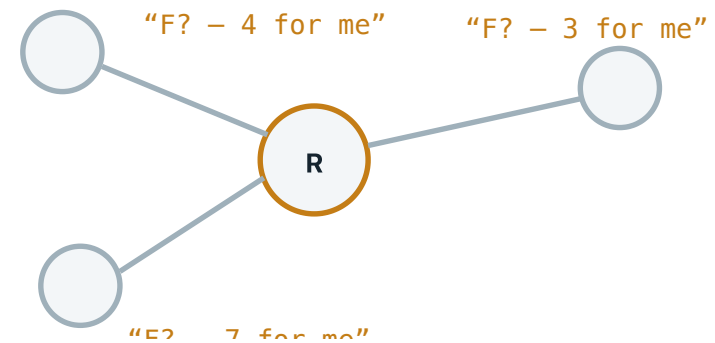
### link-state · “global”



Every router learns the **entire topology**, then computes paths alone.

Needs: a way to spread the map (flooding!) · Dijkstra

### distance-vector · “decentralised”



No map. Ask each neighbour “*how far is F for you?*” — add the link, take the minimum.

Needs: only talking to neighbours · Bellman–Ford

— These two philosophies are the next two lectures in order. Both are \*adaptive\*; they differ in what each router is allowed to know. Foreshadow: global knowledge costs messages; local knowledge costs convergence time — and worse (Wednesday's rumour problem).

## Classify this protocol

A router collects the knowledge it needs **only from directly-attached neighbours**, and updates whenever their reports change.

A — global & static

B — global & adaptive

C — decentralised & adaptive

D — decentralised & static

Neighbours-only knowledge  $\Rightarrow$  decentralised; reacting to change  $\Rightarrow$  adaptive. That's the distance-vector family.

Reveal

— 60 seconds; sets up Wednesday's vocabulary.

## CHECKPOINT

# Three questions you can now answer

Why can't static routing run the Internet — and what three jobs must replace it?

(discover · compute · react)

What is an autonomous system, and why does *who-pays-whom* split routing into interior and exterior worlds?

Why is routing “just” least-cost paths on a weighted graph — and who decides what the weights mean?

**Next:** a router that knows the whole map — **Dijkstra's algorithm**, run step by step until it becomes mechanical.

≈51 min

Close by connecting back to the opening traceroute: every hop we saw was a forwarding decision written by an adaptive protocol — by Wednesday they can name which family wrote each one.



## Going deeper



Kurose & Ross, *Computer Networking*, 8th ed. (2021) — §5.1–5.2



Stallings, *Data and Computer Communications*, 10th ed. — ch. 19.1



Optional wander: RFC 791 (1981) — four pages in, you'll meet the quote from today



This page stays up: revisit the path explorer, and the graph will follow us into the next section.

Point out the page remembers scroll position — reopening the link resumes where we stopped.

Lecture 2 of 3

# Link-State Routing: Dijkstra's Algorithm

---

Give every router the whole map, then compute shortest paths with Dijkstra's algorithm — built up step by step, on the same six-router network, until it becomes mechanical.

1 · The Routing Problem

**2 · Link-State Routing**

3 · Distance-Vector & the Real Internet

content in preparation

Waypoint "lec2". Full content lands in the next build.

Lecture 3 of 3

# Distance-Vector Routing & the Real Internet

---

Routing by rumour with Bellman–Ford — why good news travels fast and bad news barely travels — then the protocols that run the real Internet: RIP, OSPF and routing hierarchy, and BGP.

1 · The Routing Problem

2 · Link-State Routing

**3 · Distance-Vector & the Real Internet**

content in preparation

Waypoint “lec3” — distance-vector plus the real-world protocols.

## ACKNOWLEDGEMENTS

# Lineage & colophon

These lecture notes modernise COSC264 material developed by **Dr Barry Wu** (2024–25), building on notes by **Dr Mengmeng Ge** (2023) and **Assoc Prof Dan Kim** (2017), University of Canterbury, and lecture notes by **Prof Aleksandar Kuzmanovic** (CS340, Northwestern University, 2005).

Refreshed 2026 · Yinuo Zhang. Built as a self-contained web page — no trackers, no external requests; works offline. Figures original unless noted; definitions follow Kurose & Ross (8e) and Stallings (10e).



End of file. These notes continue to grow in place — same URL all term.